67. Цикл while

На этом уроке мы детально рассмотрим цикл while, его конструкцию, особенности и использование.

Оглавление:

1. Цикл while
2. Бесконечные циклы
3. Счетчик цикла while
4. Итерации
5. Вложенные циклы while
6. Тест

Цикл while

Цикл while является самым простым из четырех циклов, которые есть в языке C++. Он очень похож на **оставление if/else:**

```
while (условие)
    тело цикла;
```

Цикл while объявляется с использованием **ключевого слова while**. В начале цикла обрабатывается условие. Если его значение является ложью (false значение значения), то тело выполняется пока цикла.

Однако, в отличие от операторов if, после выполнения тела цикла, управление возвращается обратно к while и процесс проверки условия повторяется. Если условие опять является true, то тело пока цикла выполняется еще раз.

Например, следующая программа выводит все числа от 0 до 9:

```
#include <iostream>

int main()
{
    int count = 0;
    while (count < 10)
    {
        std::cout << count << " ";
        ++count;
    }

    std::cout << "done";

    return 0;
}
```

Результат выполнения программы:

```
0 1 2 3 4 5 6 7 8 9 done!
```

Рассмотрим детально эту программу. По первым, инициализируется переменная int count = 0;. Условие 0 < 10 имеет значение true, поэтому выполняется тело цикла. В первом счетчике мы выводим 0, а во втором — выполняем **инкремент** переменной count. Затем управление возвращается к началу цикла while для повторной проверки условия. Условие 1 < 10 имеет значение true, поэтому тело цикла выполняется еще раз. Тело цикла будет повторно выполняться до тех пор, пока переменная count не будет равна 10, только в том случае, когда результат условия 10 < 10 будет false, цикл завершится.

Тело цикла while может и вообще не выполняться, например:

```
#include <iostream>

int main()
{
    int count = 15;
    while (count < 10)
    {
        std::cout << count << " ";
        --count;
    }

    std::cout << "done";

    return 0;
}
```

Условие 15 < 10 сразу принимает значение false, и тело цикла пропускается. Единственное, что выведет эта программа:

```
done!
```

Бесконечные циклы

С другой стороны, если условие цикла всегда принимает значение true, то и сам цикл будет выполняться бесконечно. Это называется **бесконечным циклом**. Например:

```
#include <iostream>

int main()
{
    int count = 10;
    while (count > 0) // count никогда не станет false
        std::cout << count << " "; // count это всегда будет положительным

    return 0; // эта строка никогда не выполнится
}
```

Поскольку переменная count не увеличивается ни единожды в этой программе, то условие count > 10 всегда будет true. Следовательно, цикл никогда не будет завершен, и программа будет постоянно выводить 10 10 10 10 10....

Мы можем преднамеренно объявить бесконечный цикл следующим образом:

```
#include <iostream>

int main()
{
    while (true) // цикл будет выполняться бесконечно

    return 0;
}
```

Единственный способ выйти из бесконечного цикла — использовать операторы **return**, **break**, **goto**, выбросить исключение или воспользоваться **функцией exit()**.

Программы, которые работают до тех пор, пока пользователь не решит остановить их, иногда преднамеренно используют бесконечные циклы вместе с оператором return, break или функцией exit() для завершения цикла. Распространенная такая практика в серверной веб-программировании, которая работает непрерывно и постоянно обслуживает веб-запросы.

Счетчик цикла while

Часто нам нужно будет, чтобы наш выполнялся определенное количество раз. Для этого обычно используется переменная в виде счетчика цикла. **Счетчик цикла** — это целочисленная переменная, которая объявляется с единственной целью: считать, сколько раз выполнялся цикл. В вышеупомянутых примерах переменная count является счетчиком цикла.

Счетчики цикла часто дают разные имена, такие как i, j, k или x. Однако это имена могут вызвать кое-какие серьезные проблемы. Если вы используете одно из этих имен в вложенном цикле, то вы можете случайно использовать переменную счетчика цикла i, j или k, то в результате получите полевому своей программы. Так как i, j, и k используются во многих именах. Следовательно, лучше использовать ii, jj, или kk в качестве имен для счетчиков. Они более уникальны, не значительно лучше найти, а они выделяется в коде. А еще лучше использовать осмысленные имена для переменных, например, count, или любое другое имя, которое предоставляет контекст использования этой переменной.

Также для счетчика цикла лучше использовать тип **signed int**. Использование unsigned int может привести к неожиданным результатам. Например:

```
#include <iostream>

int main()
{
    unsigned int count = 10;

    // Countdown
    while (count >= 0)
    {
        std::cout << count << " ";
        --count;
    }

    std::cout << "Blastoff!";

    return 0;
}
```

Взгляните на эту программу еще раз и постарайтесь найти ошибку. Она начинается с вывода 10 9 8 7 6 5 4 3 2 1. Blastoff!, как и предполагалось, но затем когда count достигает значения 0, переменная count становится 4294967295. Почему? Потому что условие цикла count >= 0 никогда не будет ложным! когда count = 0, то и условие 0 >= 0 имеет значение true, выводится Blastoff!, а затем выполняется декремент переменной count, происходит **переполнение** и значение переменной становится 4294967295. И так как условие 4294967295 >= 0 является истинным, то программа продолжает свои выполнение. А поскольку счетчик цикла является типа unsigned, то он никогда не сможет быть отрицательным, а так как он никогда не сможет быть отрицательным, то цикл никогда не завершится.

Правило: Всегда используйте тип signed int для счетчиков цикла.

Итерации

Каждое выполнение цикла называется **итерацией** (или **повторение**).

Поскольку тело цикла обычно является **блоком**, и поскольку этот блок выполняется по новой с каждым повтором, то любые переменные, объявленные внутри тела цикла, создаются, а затем и уничтожаются по новой. В следующем примере переменная x создается и уничтожается 8 раз:

```
#include <iostream>

int main()
{
    int count = 1;
    while (count < 10)
    {
        // Переменная x создается при каждом повторении цикла
        int x = 0;
        std::cout << "Enter integer #" << count << ": ";
        std::cin >> x;

        result = x;

        // Переменная x уничтожается при каждом повторении цикла

        ++count;
    }

    std::cout << "The sum of all numbers entered is: " << result;

    return 0;
}
```

Для фундаментальных типов переменных это нормально. Для нефундаментальных типов переменных (таких как **структуры** или классы) это может оказаться на производительности. Следовательно, нефундаментальные типы переменных лучше определять перед циклом.

Обратите внимание, переменная count объявлена вне тела цикла. Это важно и необходимо, поскольку нам нужно, чтобы наше значение переменной сохранялось на протяжении всего выполнения цикла, а не уничтожалось и создавалось с каждым повторным циклом.

Иногда нам может понадобиться выполнить что-то при достижении определенного количества итераций, например, вставить символ новой строки. Это легко осуществить, используя оператор остатка от деления со счетчиком цикла:

```
#include <iostream>

int main()
{
    int count = 1;
    while (count < 50)
    {
        // Если count делится на 10 (после каждых десяти добавлений 10)
        if (count % 10 == 0)
            std::cout << "0" << count << "\n";
        // Иначе
        else
            std::cout << count << " ";

        // Если count равно 50, то мы закончили, не нужно выводить символ новой строки
        if (count != 50)
            std::cout << " ";

        // Увеличиваем значение счетчика цикла на единицу
        ++count;
    }

    return 0;
}
```

Результат выполнения программы:

```
01 02 03 04 05 06 07 08 09 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
```

Вложенные циклы while

Также один цикл while могут быть вложены внутри других циклов while. В следующем примере внутренний и внешний циклы имеют свои собственные счетчики. Однако, обратите внимание, условие внутреннего цикла использует счетчик внешнего цикла:

```
#include <iostream>

int main()
{
    int outer = 1;
    while (outer <= 5)
    {
        int inner = 1;
        while (inner <= outer)
            std::cout << inner << " ";

        // Печатаем новую новой строки в конце каждого ряда
        std::cout << "\n";
        ++outer;
    }

    return 0;
}
```

Результат выполнения программы:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Тест

Задание №1

Почему в программе, приведенной выше, переменная inner объявлена внутри блока while, а не сразу после объявления переменной outer (вне блока while)?

Задание №2

Напишите программу, которая выводит буквы английского алфавита от a до z вместе с кодами из ASCII-таблицы.

> **Подсказка:** Чтобы выводить символы как целые числа — используйте оператор static_cast.

Задание №3

Измените программу из последнего подраздела «Вложенные циклы while», чтобы она выводила следующее:

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

Задание №4

Теперь сделайте так, чтобы цифры выводились следующим образом (используя программу из предыдущего задания):

```
    1
  2 1
3 2 1
4 3 2 1
5 4 3 2 1
```

> **Подсказка:** Разберитесь сначала, как вывести числа следующим образом:
>
> ```
> 0 0 0 0 1
> 0 0 0 2 1
> 0 0 3 2 1
> 0 4 3 2 1
> 5 4 3 2 1
> ```

Ответ №1

Переменная inner объявлена внутри блока while так, чтобы она была восстановлена (и повторно инициализирована значением 1) каждый раз, когда выполняется внешний цикл. Если бы переменная inner была объявлена вне внешнего цикла while, то ее значение никогда не было бы сброшено до 1, или как бы прошлось это сделать самостоятельно с помощью операции присваивания. Кроме того, поскольку переменная inner используется только внутри внешнего цикла while, то имеет смысл объявить ее именно там. Помните, что переменные нужно объявлять максимально близко к их первому использованию!

Ответ №2

```
#include <iostream>

int main()
{
    char mychar = 'a';
    while (mychar <= 'z')
    {
        std::cout << mychar << " " << static_cast<int>(mychar) << "\n";
        ++mychar;
    }

    return 0;
}
```

Ответ №3

```
#include <iostream>

int main()
{
    int outer = 5;
    while (outer > 1)
    {
        int inner = outer;
        while (inner >= 1)
            std::cout << inner << " ";

        // Печатаем новую новой строчку в конце каждого ряда
        std::cout << "\n";
        --outer;
    }

    return 0;
}
```

Ответ №4

```
#include <iostream>

int main()
{
    // Цикл с 5 до 0
    int outer = 1;

    while (outer <= 5)
    {
        // Цикл с inner инициализ. в порядке убывания, поэтому цикл начинаем с 5 и до 1
        int inner = 5;

        while (inner >= 1)
        {
            // Если inner больше, чем outer, то выводим пробел
            // Иначе выводим число
            if (inner > outer)
                std::cout << inner << " ";
            else
                std::cout << " " // Оставляем дополнительный пробел

            --inner;
        }

        // Вывод для каждой переходим к следующему
        std::cout << "\n";

        ++outer;
    }

    return 0;
}
```